Started on	Wednesday, 16 June 2021, 12:45 PM
State	Finished
Completed on	Wednesday, 16 June 2021, 1:20 PM
Time taken	35 mins 38 secs
Grade	**5.20** out of 6.00 (**87**%)

Question **1**

Complete

Mark 1.00 out of 1.00

Fie un graf neorientat complet cu N noduri. Cum putem selecta un subset de muchii astfel încât să existe un drum între oricare două noduri și suma costurilor muchiilor să fie minimizată?

Construim un Arbore Minim de Acoperire cu ajutorul algoritmului Prim (ce functioneaza pe grafuri neorientate). Graful fiind complet, AMA va contine toate nodurile grafului, iar AMA ne asigura ca suma costurilor muchiilor va fi minima.

Complexitate: O(E logV) daca folosim un heap binar si O(VlogV + E) daca folosim un heap fibonacci.

Comment:

Question **2**

Complete

Mark 1.00 out of 1.00

Descrieți un algoritm pentru generarea tuturor grafurilor neorientate cu N noduri pentru care toate gradele externe sunt impare.

Generam toate grafurile neorientate cu N noduri folosing un backtracking pe stiva.

- Adaugam in solutie doar nodurile din graf ce au grad extern impar.
- Printam solutia in momentul in care am adaugat un numar de N noduri.

Complexitate: O(V ^ N) (unde V = nr noduri), pentru ca avem N pozitii ce trebuie umplute in solutie si avem un domeniu de dimensiune V. Pe prima pozitia din graful cu N noduri avem de ales dintre V noduri, pe a doua pozitie tot din V noduri etc => complexitate V * V * ... V de N ori.

Comment:

Question **3**

Complete

Mark 0.20 out of 1.00

Cum se poate calcula eficient drumul minim între un nod sursă și un nod destinație într-un graf neorientat aciclic în care toate muchiile au costuri negative ?

Pentru ca toate muchiile au costuri negative, in momentul in care operam cu costul unei muchii, putem sa o inmultim cu -1 (pentru a obtine cost pozitiv), iar atunci cand dorim sa extragem un rezultat (ex.: costul drumului minim cerut de la src la dest), inmultim rezultatul stocat cu -1 (spre ex: in vectorul de distante avem ca v[dest] = 10. Putem inmulti acel 10 cu -1 si obtinem -10, exact cat ar fi fost suma costurilor muchiilor in graful initial (-1 + -2 + -3 + -4 = -10, sa zicem)

Folosim un algoritm de tip Dijkstra ce gaseste drumurile minime de la src la toate destinatiile posibile (inclusiv la destinatia ceruta). Pentru ca, in urma artificiului descris mai devreme, nu va mai fi nicio muchie cu cost negativ, Dijkstra va functiona fara probleme si va gasi drumul de cost minim de la src la dest.

Complexitate: O(E log V) daca folosim un priority queue sau O(V logV + E) daca folosim un heap fibonacci. Legat de prority queue, Djikstra, fiind Greedy, selecteaza la fiecare pas nodul cu distanta minima fata de sursa la care sa se duca. Aceasta alegere se face cu ajutorul acelui priority queue, de unde vine complexitatea de E log V.

Comment:
Există un singur drum între oricare două noduri: orice parcurgere (BFS/DFS) va întoarce soluția dorită

Insa apreciez explicatiile 🙂

Totusi, minimul unor numere negative va deveni maximul numerelor atunci cand le inmultesti cu -1

Question **4**

Complete

Mark 1.00 out of 1.00

Se dă un graf orientat cu n noduri, având capacități numere prime pe muchii. Se dau K surse și o destinație unică T. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la toate sursele la destinație.

Putem folosi un Edmonds-Karp adaptat pentru surse si destinatii multiple. Reducem graful initial la unul pentru Edmonds-Karp cu single src si single dest prin adaugarea unei "metasurse" (= noduri) conectata printr-o muchie de capacitate nelimitata cu sursele mici (initiale, cele de la care vrem fluxul pana la dest-urile multiple); in acelasi sens vom conecta "metadestinatii" (= noduri) prin muchii cu capacitati infinite cu destinatiile mici, initiale.

Complexitate: O(E^2 * V) (cat era edmonds-karp initial) * O(V) (pentru ca putem avea pana la V surse) = O (E^2 * V^2).

Comment:
Corect

Question **5**

Complete

Mark 1.00 out of 1.00

Cum putem găsi eficient cel mai scurt drum simplu, intre doua noduri S și D, într-un graf orientat, dacă toate arcele au costuri pozitive mai puțin cele care ies din sursă.

Daca prin "drum scurt si simplu" intelegem doar sa fie nr minim de noduri intermediare, folosim un BFS de complexitate O(V * E) si tinem cont de cost pe masura ce construim drumul. Primul drum de la src la dest furnizat de BFS va fi si cel mai scurt (1 -> 2 -> dest mai scurt decat 1 -> 2 -> 3 -> ... -> dest)

Daca prin "drum scurt si simplu" intelegem drum de cost minim, folosim un Bellman-Ford, el fiind capabil sa furnizeze un rezultat corect chiar si cu arce cu costuri negative. Complexitate: O(V * E); el facand V - 1 iteratii a cate E relaxari, deci O(E * (V - 1)) deci O(E * V).

Comment:
Drum simplu = drum care contine un nod o singura data

Pentru aceasta problema, Bellman-Ford este ineficient, este un graf particular pe care putem apela Dijkstra.

Question **6**

Complete

Mark 1.00 out of 1.00

Determinați dacă un graf este un graf bipartit (mulțimea nodurile se poate împărți în două submulțimi disjuncte).

Folosim un BFS. Parcurgem graful si punem "etichete" peste noduri in functie de partitatea nivelului la care a fost gasit nodul respectiv in parcurgere (spre exemplu: punem pune un "0" peste nodudrile de nivel par si un "1" peste nodurile de pe nivel impar). Spre exemplu, nodurile sursa, fiind la nivel 0, va avea atribuit "0" iar nodurile gasite la primul pas din BFS vor avea atribuite "1".

La final, nodurile ce aveau atribuite "0" vor reprezenta prima submultime, iar nodurile cu "1" vor reprezentata cea de-a doua submultime.

Complexitate: O(V * E), fiind bazat pe BFS.

Comment:
Complexitatea BFS este O(|V| + |E|)

◄ Examen PA - Subiect I - part1 (raspuns direct) - 16.06.2021

Jump to...

Examen PA - Subiect II - 16.06.2021 ►